steering methods, which modify intermediate hidden states during generation, our approach modifies the cached keys and values associated with specific tokens, enabling a one-shot intervention that can be precomputed and reused. This technique is compatible with standard inference APIs and does not require model fine-tuning or prompt engineering.

3.1 PRELIMINARIES

Transformer-based language models rely on the self-attention mechanism, which operates on sets of query, key, and value vectors to compute contextualized token representations. For a given input sequence, the attention output at layer l is computed as:

$$\text{Attention}(\boldsymbol{Q}^l, \boldsymbol{K}^l, \boldsymbol{V}^l) = \text{softmax}\left(\frac{\boldsymbol{Q}^l(\boldsymbol{K}^l)^\top}{\sqrt{D_h}}\right)\boldsymbol{V}^l$$

where $\boldsymbol{Q}^l, \boldsymbol{K}^l, \boldsymbol{V}^l \in \mathbb{R}^{T\times H\times D_h}$ are the query, key, and value tensors at layer l, T is the sequence length, H is the number of attention heads, and D_h is the dimensionality of each head.

During autoregressive decoding, the model stores the keys $\boldsymbol{K}^l$ and values $\boldsymbol{V}^l$ corresponding to previously processed tokens, which is known as a key-value (KV) cache. These cached tensors are used to efficiently compute attention for each new token without recomputing representations for the entire sequence. Importantly, these cache entries can be precomputed and reused across multiple examples (such as caching a system prompt), which is especially useful in scenarios involving large models or repeated inference over similar inputs. This makes the KV cache a potential target for behavioral interventions, offering compatibility with real-world settings.

3.2 EXTRACTING KEY-VALUE STEERING VECTORS

Similarly to activation steering, we construct a contrastive set of prompt pairs $C = \{(p_0^+, p_0^-), (p_1^+, p_1^-), \ldots, (p_N^+, p_N^-)\}$ to extract the key-value steering vectors. We refer to prompts that demonstrate the desired behavior as positive and the prompts without such behavior as negative. We discuss the details of how the positive and negative prompts are constructed for the reasoning induction task in Section 3.5.

For each contrastive pair of examples, we make a forward pass and extract the keys and values vectors from the designated token position (typically the final token of the input prompt). The vectors are then aggregated using the Mean-of-Differences method:

$$\boldsymbol{S}_l^k = \frac{1}{N}\sum_{(p^+,p^-)\in C} f_l(p^+) - f_l(p^-) \qquad \boldsymbol{S}_l^v = \frac{1}{N}\sum_{(p^+,p^-)\in C} f_l(p^+) - f_l(p^-)$$

where f_l is a Transformer layer, $\boldsymbol{S}_l^k \in \mathbb{R}^{H\times D_h}$ and $\boldsymbol{S}_l^v \in \mathbb{R}^{H\times D_h}$ are the resulting steering tensors at layer l, with H denoting the number of attention heads and D_h their dimension. By taking the difference between positive and negative examples and averaging across multiple contrastive pairs, we aim to isolate a directional signal associated with target behavior while minimizing the amount of noise introduced by information from individual examples.

3.3 APPLYING KEY-VALUE STEERING VECTORS

At inference time, we perform a standard forward pass on the input prompt to populate the KV cache. Then, at each layer l, we modify the cached key and value vectors at a target token position of the KV cache as follows:

$$\boldsymbol{V}_l^* = \boldsymbol{V}_l + c^v \boldsymbol{S}_l^v \qquad \boldsymbol{K}_l^* = \boldsymbol{K}_l + c^k \boldsymbol{S}_l^k$$

where $\boldsymbol{K}_l, \boldsymbol{V}_l \in \mathbb{R}^{H\times D_h}$ are the original cached key and value vectors at layer l, and $\boldsymbol{S}_l^k, \boldsymbol{S}_l^v \in \mathbb{R}^{H\times D_h}$ are the steering vectors, and $c^k, c^v \in \mathbb{R}$ are scalar coefficients controlling the steering strength. Then the generation proceeds as usual using the modified cache.

3.4 ELIMINATING STEERING AMPLIFICATION

Cache steering differs fundamentally from traditional activation steering in *how* and *when* interventions are applied. Activation steering can amplify across layers and timesteps, making it unstable and sensitive

to hyperparameters. Cache steering eliminates this amplification by modifying the fixed KV cache once after prefilling, as illustrated in Figure 1. Below, we outline the core intuition behind this contrast.

At a specific timestep t, activation steering explicitly affects the current hidden state at a chosen layer l. This modification propagates both *vertically* through all subsequent layers $l+1$ to $l+N$ and *horizontally* into future tokens as decoding continues. Because interventions accumulate across layers and timesteps, small changes can compound into "oversteering," which can negatively affect generation quality. This makes the method highly sensitive to hyperparameters such as steering strength and application layer (Turner et al., 2024; Lee et al., 2025; Wang et al., 2025; Dong et al., 2024; Rimsky et al., 2024; Wang et al., 2024; Stolfo et al., 2025; Zhang & Viteri, 2025; Postmus & Abreu, 2024).

In contrast, cache steering modifies the *fixed* key and value representations of *past* tokens after the prefilling stage. These cached representations are no longer transformed through the network and can therefore be adjusted vertically across all layers without risk of compounding. Future tokens then attend to this modified cache, so the steering effect propagates horizontally across the tokens during decoding. This one-shot intervention avoids cascading effects, allowing cache steering to be both stable to hyperparameters and efficient at runtime.

In short, cache steering replaces the compounding per-step interventions of activation steering with a single post-prefill modification that avoids amplification, yielding a stable and efficient mechanism for guiding model behavior.

3.5 Implementation details

Contrastive set construction. To extract steering vectors, we construct a contrastive dataset consisting of paired prompts. Each pair includes a **positive example** (containing explicit chain-of-thought reasoning) and a **negative example** (containing only the final answer). Each contrastive prompt is created using few-shot in-context learning (ICL) examples. Specifically, both the positive and negative prompts include n ICL examples followed by a question and a generation prompt. The positive and negative prompts differ only in the presence of reasoning steps in the ICL examples (see Appendix C.1 for more details and an illustrative example).

Extraction and application positions. We extract key and value vectors from the final token of the prompt, which typically corresponds to the last token of the generation prompt depending on the model's chat template (e.g. `"\n\n"` in `"assistant\n\n"`). During inference, we aim to apply cache steering to the same logical position in the prompt as used during extraction. However, due to the autoregressive decoding mechanism (see Section 3.1), the KV cache is populated only after each token is processed. To ensure alignment, we append a neutral offset token (e.g., a newline or whitespace) to the prompt, so that the KV representation of the final token can be used in the generation of the next tokens. This ensures the cache steering affects the intended location. Details on token alignment and cache offset are provided in Appendix C.5.

Hyperparameters. As with activation steering, the steering strength coefficients of key and value vectors are treated as hyperparameters. Since we are interested in distilling reasoning behaviors from larger models, we additionally treat the number of contrastive pairs and the number of in-context examples in each pair as additional hyperparameters. Similarly to other steering approaches, we perform a small grid search over the hyperparameters to obtain reasonable values for each model-dataset pair (Turner et al., 2024; Lee et al., 2025; Wang et al., 2025; Dong et al., 2024; Rimsky et al., 2024; Wang et al., 2024; Stolfo et al., 2025; Zhang & Viteri, 2025; Postmus & Abreu, 2024). We find that steering coefficients tend to lie within consistent ranges across tasks, suggesting robustness in the method's behavior. More on this in Section 5.3. The full list of hyperparameters can be found in Appendix G.

4 Experimental Setup

Datasets. We use four common reasoning benchmarks for the evaluation: GSM8K (Cobbe et al., 2021), CommonsenseQA (Talmor et al., 2018), ARC-Challenge (Clark et al., 2018), and PIQA (Bisk et al., 2020). These datasets span arithmetic reasoning, commonsense inference, scientific questions, and physical commonsense reasoning. For each dataset, we generate elaborate step-by-step answers to a subset of questions from the corresponding training sets using GPT-4o, which are then used in

Table 1: **Comparison of baselines, activation steering, and cache steering on four reasoning benchmarks**. We evaluate six models of different sizes on GSM8K, ARC-Challenge, CSQA, and PIQA using both greedy decoding (left block) and sampling-based decoding (right block). Results show that cache steering consistently improves reasoning performance, often outperforming both baseline and activation steering. Combining cache steering with CoT prompting yields further gains in more than half of the cases. Numbers in parentheses denote standard deviation across 5 sampled generations per input; the sampling block highlights that cache steering produces a *stable shift in logits*, as reflected by consistently better or on-par performance under stochastic decoding.

Dataset	Model	Greedy					Sampling	
		Baseline	CoT prompt	Activation steering	Cache steering	Cache steering + CoT prompt	Baseline	Cache steering
ARC-c	SmolLM2-360M	24.32	26.62	24.06	**27.13**	25.26	24.16 (1.13)	**24.52** (0.87)
	Llama-3.2-1B	53.67	53.75	53.84	55.03	**56.14**	52.29 (0.81)	**53.16** (1.44)
	Llama-3.2-3B	74.32	77.13	74.23	79.27	**79.52**	74.64 (0.36)	**77.71** (0.82)
	Qwen2-0.5B	39.51	37.20	**40.69**	40.36	38.82	**38.05** (0.21)	35.96 (0.93)
	Llama-3.1-8B	83.11	84.98	84.64	**85.58**	85.24	82.66 (0.28)	**85.09** (0.64)
	Phi-4-mini	84.56	86.69	86.18	**87.97**	86.77	83.46 (0.56)	**87.2** (0.62)
GSM8K	SmolLM2-360M	8.49	**10.39**	7.66	8.95	**10.39**	**8.08** (0.38)	7.87 (0.4)
	Llama-3.2-1B	45.56	46.10	45.41	46.32	**47.16**	43.71 (0.83)	**43.88** (1.22)
	Llama-3.2-3B	68.54	71.57	68.38	67.17	**72.10**	**68.22** (0.43)	67.57 (1.22)
	Qwen2-0.5B	17.44	24.94	23.81	18.04	**25.47**	**16.94** (1.08)	16.48 (0.4)
	Llama-3.1-8B	76.34	77.56	76.50	75.81	**77.86**	**75.94** (0.62)	75.22 (0.58)
	Phi-4-mini	77.94	74.68	75.89	**78.47**	75.74	**77.48** (0.62)	77.1 (0.66)
CSQA	SmolLM2-360M	19.74	22.11	19.66	21.95	**22.52**	20.02 (1.77)	**21.31** (0.67)
	Llama-3.2-1B	53.56	54.71	54.14	**55.20**	53.56	**51.45** (0.64)	50.78 (0.73)
	Llama-3.2-3B	70.27	**72.56**	69.12	72.32	72.48	70.09 (0.78)	**70.40** (1.1)
	Qwen2-0.5B	**47.42**	44.31	45.95	46.03	45.37	**45.67** (1.18)	42.36 (1.11)
	Llama-3.1-8B	73.87	74.04	73.30	**75.27**	74.37	73.92 (0.37)	**74.27** (1.01)
	Phi-4-mini	69.78	70.11	69.29	70.00	**70.52**	**68.22** (0.62)	67.52 (1.08)
PIQA	SmolLM2-360M	50.38	**52.61**	49.62	51.31	52.50	48.12 (0.66)	**50.72** (1.03)
	Llama-3.2-1B	65.29	64.96	61.48	63.76	**66.43**	**65.02** (0.88)	62.48 (1.57)
	Llama-3.2-3B	69.42	**76.93**	72.31	73.34	76.88	68.35 (0.28)	**71.83** (0.66)
	Qwen2-0.5B	52.12	53.43	53.43	54.57	**55.55**	51.82 (0.68)	**53.86** (0.44)
	Llama-3.1-8B	80.03	81.61	80.25	82.86	**83.13**	79.08 (0.39)	**83.41** (0.52)
	Phi-4-mini	78.29	79.59	**80.74**	79.33	80.25	77.19 (0.63)	**79.46** (0.68)

positive examples in the contrastive set. The details on the specific prompt used to generate these steps and the generation procedure can be found in the Appendix C.6. Steering vectors are computed using the training set, while evaluation is performed on the corresponding test sets.

Models. We evaluate cache steering on small instruction-tuned models from four families: Llama-3.2 (1B and 3B variants), SmolLM2 (360M), Qwen2 (0.5B), and Phi-4-mini (3.8B) (Grattafiori et al., 2024; Team, 2024; Allal et al., 2025; Abouelenin et al., 2025). Additionally, we add the Llama-3.1 (8B) model to evaluate how cache steering scales beyond the smallest models. The list with the full model names and URLs can be found in Appendix C.8.

Decoding strategies. Since cache steering affects internal representations, which result in a shift in output logits, we evaluate our approach using both deterministic and stochastic decoding. For sampling-based decoding, we assess the consistency of steering effects by generating the response with 5 different seeds and comparing that to the baseline generations using the same setup. The generation arguments can be found in Appendix C.4.

Answer extraction and metrics. Answer correctness is determined using task-specific heuristics. For GSM8K, we extract the final number mentioned in the output using digit pattern matching (Wang et al., 2023; Wang & Zhou, 2024). For multiple-choice tasks (ARC, PIQA, CSQA), we develop a 4-stage extraction pipeline that uses soft string matching against known answer choices, with failover to constrained decoding. More details on the answer extraction process can be found in Appendix C.2.